

Introduction to Julia

Leonardo De Carlo, Giosuè Muratore

TecWeb 2026

Motivating problem

Gromov–Witten invariants

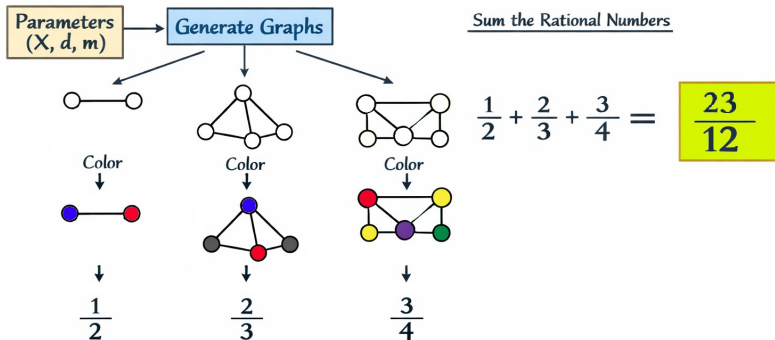
In Algebraic Geometry, there is a formula devised by Michael Atiyah, Raoul Bott and Maksim Končević, which allows to compute Gromov–Witten invariants. Those are *intensively studied* rational numbers that depend on parameters.

This formula works in the following way:

1. Choose your parameters (X, d, m) .
2. Generate some graphs and other attributes (colorations, etc...), that depend on the parameters.
3. Associate to each of those objects a rational number.
4. Sum all the rational numbers obtained.

The result of the sum is the Gromov–Witten invariant you were searching for.

ABK formula



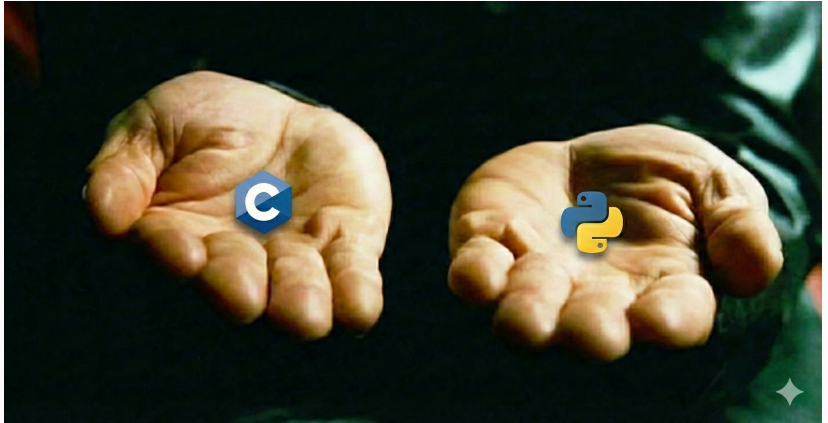
The Atiyah–Bott–Konevič formula is very powerful as it covers a lot of invariants. Unfortunately, it is very inefficient: The number of graphs grows exponentially with the parameters, making the computation of those invariants computationally expensive.

My goal was to create the *first* package for computation of GW invariants for **any** set of parameters. I needed a language that:

- It is free.
- It can already manage efficiently ~~float numbers~~ rational numbers and other mathematical objects, like polynomials.
- It is easy to use.
- It is fast.

So, I had a choice:

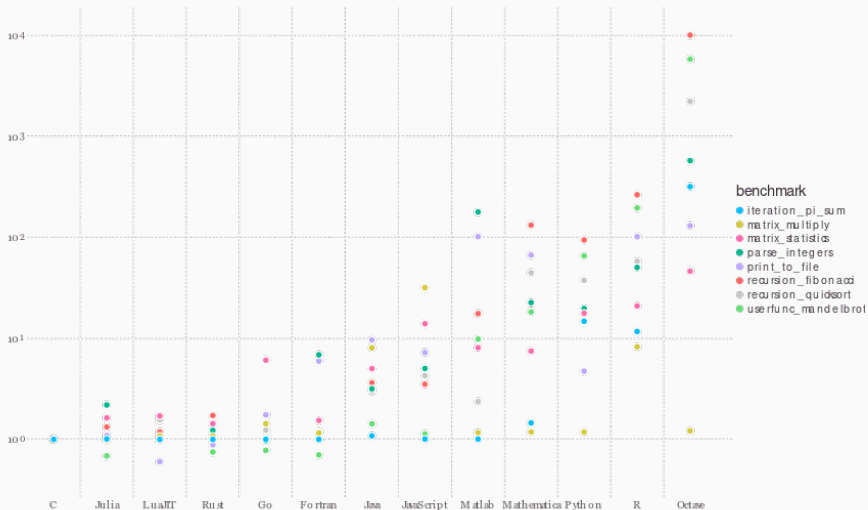
The Choice



Eventually, I chose **Julia**. It is a modern dynamic general-purpose programming language. It is especially indicated for numerical analysis tasks, but:

- It is open source and very easy to learn, with C-like instructions.
- It supports other source codes: C and Fortran (natively), C++, Python, R, etc (through wrappers),
- It is designed for modern applications: GPU computations, multithreading, machine learning,...
- It has a very powerful package manager.
- A very active Julia community produces new packages every day, closing the gap with classical languages.

Is Julia fast?



Multiple dispatch

One of the reasons Julia is so fast is *multiple dispatch*.

Suppose you want to define a Python code that computes the *area* of some geometric figures.

Multiple dispatch

First, we define the classes of some geometric figures:

```
1 import math
2
3 class Circle:
4     def __init__(self, radius):
5         self.radius = radius
6
7 class Rectangle:
8     def __init__(self, width, height):
9         self.width = width
10        self.height = height
```

Multiple dispatch

Second, we define our function area:

```
1 def area(shape):
2
3     if isinstance(shape, Circle):
4         return math.pi * shape.radius ** 2
5
6     elif isinstance(shape, Rectangle):
7         return shape.width * shape.height
8
9     else:
10        raise NotImplementedError
```

Multiple dispatch

Our code is not performant:

- multiple calls of `isinstance`,
- difficult to write specialized code,
- long conditional chain, difficult to read,
- adding a new shape (for example, `Triangle`) implies modifying the function `area`.

Let us see how Julia deals with it.

Multiple dispatch

```
1 abstract type GeometricObject end
2 abstract type Shape{T <: Number} <: GeometricObject end
3
4 struct Circle{T <: Number} <: Shape{T}
5     radius::T
6 end
7
8 struct Rectangle{T <: Number} <: Shape{T}
9     width::T
10    height::T
11 end
```

Multiple dispatch

```
1 area(shape::Circle) = π * shape.radius^2
2 area(shape::Rectangle) = shape.width * shape.height
3 area(shape::Shape) = error("Area not implemented")
```

This code is much more readable (no ifs), moreover the **compiler** chooses the fittest code in no time.

Multiple dispatch

Adding a new type requires minimal code additions:

```
1 struct Triangle{T <: Number} <: Shape{T}
2     base::T
3     height::T
4 end
5
6 area(t::Triangle) = t.base * t.height / 2
```

This improves code reusability and speed at runtime.